

" به نام خدا "



دانشکده فنی مهندسی

رشته مهندسی کامپیوتر

گرایش هوش مصنوعی و نرم افزار

عنوان :

گزارش اول پروژه نهایی مباحث ویژه

دانشجویان :

محمدرضا باقرنژادمیری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

استاد :

دکتر مهدیه قاسمی

فهرست

1. مستندات فنی.....	4
1-1 گزارش تخصصی و راهنمای جامع سیستم کنترل نسخه Git	5
DVC 2-1.....	11
Docker 3-1.....	17
prefect 4-1.....	23
2. صورت جلسات.....	30
2-1 صورت جلسه هماهنگی پروژه MLOps (جلسه اول- هفته اول)	31
2-2 صورت جلسه هماهنگی پروژه MLOps (جلسه دوم و سوم- هفته اول).....	33

1. مستندات فنی

1-1 گزارش تخصصی و راهنمای جامع سیستم کنترل نسخه Git

۱. مقدمه و تاریخچه

در دنیای توسعه نرم افزار، مدیریت تغییرات کد (Version Control) یکی از حیاتی ترین ارکان است. **Git** یک سیستم کنترل نسخه توزیع شده (Distributed Version Control System) است که امروزه به استاندارد مطلق صنعت نرم افزار تبدیل شده است.

- **سازنده و سال انتشار:** گیت در سال ۲۰۰۵ توسط لینوس توروالدز (خالق لینوکس) ابداع شد. توروالدز پس از آنکه دسترسی جامعه لینوکس به ابزار BitKeeper قطع شد، در مدت کوتاهی گیت را با تمرکز بر سرعت و امنیت طراحی کرد.
- **هدف از ایجاد:** هدف اصلی، جایگزینی سیستم های قدیمی مثل SVN بود که متمرکز (Centralized) بودند و در پروژه های بزرگ با کندی و اختلال مواجه می شدند.
- **سیر تکاملی:** گیت از یک ابزار پیچیده خط فرمان برای مدیریت هسته لینوکس، به ابزاری تبدیل شد که امروزه زیربنای پلتفرم های عظیمی مثل GitHub و GitLab را تشکیل می دهد و تقریباً ۹۰٪ توسعه دهندگان جهان از آن استفاده می کنند.

۲. توضیحات فنی و معماری

برخلاف سیستم های قدیمی که تغییرات را به صورت "تفاوت فایل ها" (Delta-based) ذخیره می کردند، گیت با مفهوم **Snapshot** کار می کند.

معماری سه لایه (The Three States)

در گیت، فایل های شما همیشه در یکی از سه وضعیت زیر قرار دارند:

1. **Working Directory** (پوشه کاری): جایی که شما فایل ها را ویرایش می کنید.

2. **Staging Area / Index** (منطقه انتظار): فایلی است که اطلاعات مربوط به تغییرات بعدی برای ثبت (Commit) را نگه می‌دارد.

3. **Git Directory / Repository** (مخزن): جایی که گیت فراداده‌ها و پایگاه داده اشیاء پروژه شما را ذخیره می‌کند (پوشه .git).

ساختار داده‌ای

گیت از یک ساختار گراف جهت‌دار بدون چرخه (**DAG - Directed Acyclic Graph**) برای مدیریت تاریخچه استفاده می‌کند. هر "Commit" در واقع یک گره در این گراف است که به والد خود اشاره می‌کند. امنیت داده‌ها نیز توسط الگوریتم‌های هش (SHA-1) تضمین می‌شود؛ به این معنا که کوچکترین تغییر در فایل، شناسه (Hash) آن را عوض می‌کند.

۳. ویژگی‌های کلیدی

چرا گیت به انتخاب اول تیم‌ها تبدیل شده است؟

- **توزیع‌شدگی (Distributed):** هر توسعه‌دهنده یک نسخه کامل از تاریخچه پروژه را روی سیستم خود دارد. این یعنی برای کار کردن (به جز Push/Pull) نیازی به اینترنت یا اتصال به سرور مرکزی نیست.
- **سیستم شاخه‌بندی (Branching) بی‌نظیر:** ساختن شاخه در گیت بسیار سریع و ارزان است. این ویژگی اجازه می‌دهد تیم‌ها روی ویژگی‌های مختلف (Features) به صورت ایزوله کار کنند.
- **سرعت بالا:** تقریباً تمام عملیات‌ها روی فایل‌های محلی انجام می‌شوند، لذا سرعت اجرای دستورات بسیار بالاست.
- **یکپارچگی داده‌ها:** به دلیل استفاده از مکانیزم هش، امکان تخریب نامحسوس فایل‌ها یا تغییر تاریخچه بدون ردپای فنی وجود ندارد.

۴. پیش‌نیازها و نصب

برای شروع کار با گیت، ابتدا باید آن را بر روی سیستم عامل خود نصب کنید.

نصب در سیستم عامل های مختلف:

- **Windows**: دانلود نصب کننده Git for Windows از سایت git-scm.com. این بسته شامل Git Bash نیز هست که محیطی شبیه لینوکس در ویندوز فراهم می کند.
- **macOS**: استفاده از دستور brew install git در ترمینال (اگر Homebrew نصب باشد) یا نصب از طریق Xcode Command Line Tools.
- **Linux (Ubuntu/Debian)**: اجرای دستور sudo apt install git.

پیکربندی اولیه (Configuration)

پس از نصب، حتماً باید هویت خود را برای گیت تعریف کنید:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "yourname@example.com"
```

```
git config --global user.email "yourname@example.com"
```

۵. راهنمای شروع سریع (Quick Start)

بیا یک پروژه ساده را با گیت مدیریت کنیم.

۱. ایجاد مخزن:

```
mkdir my-project
```

```
cd my-project
```

۲. ساخت فایل و بررسی وضعیت:

```
echo "Hello Git" > README.md
```

۳. ثبت تغییرات (The First Commit):

```
git add README.md
```

```
git commit -m "Initial commit: Add README file"
```

توضیح خط به خط:

- `git init`: پوشه را به یک مخزن گیت تبدیل می‌کند.
- `git add`: به گیت می‌گوید که می‌خواهیم این فایل خاص در نسخه بعدی (Snapshot) گنجانده شود.
- `git commit`: یک عکس از وضعیت فعلی Staging Area می‌گیرد و با یک پیام توضیحی (m-) ذخیره می‌کند.

۶. مفاهیم و دستورات اساسی

در جدول زیر، دستوراتی که هر روز با آن‌ها سر و کار خواهید داشت آورده شده است:

دستور	عملکرد
<code>git clone [url]</code>	کپی کردن یک مخزن موجود از سرور (مثل GitHub) به سیستم محلی
<code>git status</code>	نمایش فایل‌های تغییر یافته و فایل‌هایی که هنوز Add نشده‌اند
<code>git log</code>	نمایش لیست تمام Commit‌های انجام شده به ترتیب زمانی معکوس
<code>git branch [name]</code>	ایجاد یک شاخه جدید برای توسعه یک ویژگی خاص
<code>git checkout [branch]</code>	سوئیچ کردن بین شاخه‌های مختلف
<code>git merge [branch]</code>	ادغام تغییرات یک شاخه در شاخه فعلی (مثلاً ادغام Feature در Main)

عملکرد	دستور
دریافت آخرین تغییرات از سرور و ادغام خودکار با کد محلی	git pull
ارسال Commit های محلی به سرور مرکزی	git push
نمایش دقیق تغییرات خط به خط در فایل ها قبل از Commit	git diff

۷. کاربردهای اصلی

گیت در سناریوهای زیر برای تیم ما حیاتی است:

1. **همکاری تیمی:** چندین برنامه نویس همزمان روی یک پروژه کار می کنند بدون اینکه کدهای یکدیگر را رونویسی (Overwrite) کنند.
2. **مدیریت نسخه های محصول:** نگهداری نسخه پایدار (Production) در یک شاخه و توسعه ویژگی های جدید در شاخه های دیگر.
3. **بازگشت به عقب (Rollback):** اگر باگی در نسخه جدید پیدا شد، می توان در عرض چند ثانیه پروژه را به آخرین نسخه سالم بازگرداند.
4. **کد ریویو (Code Review):** استفاده از Pull Request ها برای بررسی کد قبل از ادغام با بدنه اصلی پروژه.

۸. مزایا و معایب (دیدگاه واقع بینانه)

مزایا:

- **پایداری:** به شدت قابل اعتماد است و احتمال از دست رفتن داده ها نزدیک به صفر است.
- **انعطاف پذیری:** از هر نوع گردش کار (Workflow) مثل GitFlow یا Trunk-based پشتیبانی می کند.

- جامعه بزرگ: برای هر مشکلی در گیت، هزاران راه حل در اینترنت وجود دارد.

معایب:

- منحنی یادگیری: یادگیری دستورات پیشرفته (مثل rebase یا cherry-pick) زمان بر است.
- مدیریت فایل های حجیم: گیت برای فایل های متنی طراحی شده؛ مدیریت فایل های باینری بسیار بزرگ (مثل ویدیو یا دیتابیس های سنگین) در آن کمی دشوار است (که با Git LFS حل می شود).

۹. اکوسیستم و منابع یادگیری

پلتفرم های میزبانی:

- **GitHub**: بزرگترین جامعه متن باز.
- **GitLab**: عالی برای تیم های سازمانی و دارای ابزارهای داخلی CI/CD.
- **Bitbucket**: یکپارچگی خوب با محصولات Atlassian مثل Jira.

ابزارهای مکمل (GUI):

اگر کار با خط فرمان برایتان سخت است، این ابزارها پیشنهاد می شوند:

- **GitKraken**: محیط گرافیکی بسیار زیبا و قدرتمند.
- **SourceTree**: ابزار رایگان و حرفه ای شرکت Atlassian.
- **VS Code Git Extension**: پلاگین داخلی ویژوال استودیو کد که بسیار کاربردی است.

منابع برای مطالعه بیشتر:

1. کتاب **Pro Git**: مرجع اصلی و رایگان که در سایت رسمی گیت موجود است.
2. **Learn Git Branching**: یک وبسایت بازی گونه و تعاملی برای یادگیری بصری شاخه ها.
3. مستندات **Atlassian Git Tutorials**: آموزش های بسیار روان برای مفاهیم تیمی.

DVC 2-1

گزارش جامع معرفی و به کارگیری ابزار (DVC (Data Version Control

۱. مقدمه و تاریخچه

در دنیای توسعه نرم افزار، ابزارهایی مانند Git استاندارد طلایی برای کنترل نسخه کد هستند. اما با ظهور هوش مصنوعی و یادگیری ماشین (Machine Learning)، مهندسان با چالش جدیدی روبرو شدند: مدیریت داده های حجیم و مدل های سنگین. گیت برای ذخیره فایل های چند گیگابایتی طراحی نشده است.

DVC یا Data Version Control در سال ۲۰۱۷ توسط شرکت Iterative.ai معرفی شد. هدف اصلی از ایجاد آن، پر کردن شکاف میان ابزارهای سنتی مهندسی نرم افزار و نیازمندی های پروژه های داده محور بود. در طول سال ها از یک ابزار ساده برای ردیابی فایل ها به یک پلتفرم کامل برای مدیریت آزمایش ها (Experiments)، خط لوله های داده (Pipelines) و مدیریت مدل تبدیل شده است.

۲. توضیحات فنی و معماری

DVC یک سیستم کنترل نسخه متن باز (Open-source) برای پروژه های یادگیری ماشین است. این ابزار بر پایه مفاهیم Git ساخته شده اما به جای کپی کردن داده ها، از "متاداده ها" (Metadata) استفاده می کند.

معماری کلی:

معماری DVC بر اساس جداسازی "کد" از "داده" است:

Git کدها و فایل های کوچک تنظیمات (.dvc) را مدیریت می کند.

DVC داده های حجیم، مدل ها و دایرکتوری ها را مدیریت کرده و آن ها را در یک Remote Storage مانند S3، Google Drive، یا سرور محلی (ذخیره می کند).

Linker: در سیستم فایل شما، DVC از Symlinks یا Hardlinks استفاده می کند تا فایل های حجیم را از کش (Cache) داخلی خود به پوشه پروژه متصل کند، بدون اینکه فضای اضافی اشغال شود.

۳. ویژگی‌های کلیدی

چرا تیم ما باید از DVC استفاده کند؟

ردیابی داده و مدل: امکان بازگشت به نسخه‌های قبلی یک دیتاست یا یک مدل آموزش دیده) همانند (git checkout

استقلال از ذخیره‌ساز: از AWS S3 ، Google Cloud Storage ، Azure Blob ، HDFS ، SSH و حتی Google Drive پشتیبانی می‌کند.

مدیریت خطلوله (Pipeline): تعریف مراحل مختلف (پیش‌پردازش، آموزش، ارزیابی) و اجرای خودکار آن‌ها در صورت تغییر در داده یا کد.

تکرارپذیری (Reproducibility): تضمین اینکه هر عضو تیم می‌تواند دقیقاً همان نتیجه‌ای را بگیرد که شما گرفته‌اید، با همان داده و همان کد.

مدیریت آزمایش‌ها: امکان مقایسه معیارهای دقت (Metrics) بین نسخه‌های مختلف پروژه.

۴. پیش‌نیازها و نصب

قبل از نصب DVC ، باید Git و Python روی سیستم نصب باشند.

نصب در سیستم عامل‌های مختلف:

```
pip install dvc
pip install "dvc[s3]"
```

در لینوکس:

```
sudo wget
https://dvc.org/deb/dvc.list -O /etc/apt/sources.list.d/dvc.list
sudo apt update
sudo apt install dvc
```

۵. راهنمای شروع سریع (Quick Start)

بیایید یک سناریوی ساده را دنبال کنیم: ردیابی یک فایل داده به نام `data.csv`.

گام ۱: مقداردهی اولیه

ابتدا در پوشه پروژه خود DVC را فعال کنید:

```
git init
dvc init
git commit -m "Initialize DVC"
```

گام ۲: اضافه کردن داده

فرض کنید فایلی به نام `data.csv` دارید. برای ردیابی آن توسط DVC:

```
dvc add data.csv
```

توضیح: با این کار فایلی به نام `data.csv.dvc` ساخته می‌شود. این فایل کم‌حجم است و باید در Git ذخیره شود. فایل اصلی `data.csv` به `.gitignore` اضافه می‌شود.

گام ۳: ذخیره در گیت

```
git add data.csv.dvc .gitignore
git commit -m "Add data tracking"
```

گام ۴: تنظیم فضای ذخیره‌سازی ابری

```
dvc remote add -d myremote /tmp/dvc_storage
dvc push
```

گام ۵: بازیابی در سیستم دیگر

یک عضو دیگر تیم برای دریافت داده‌ها کافیسست دستور زیر را بزنند:

```
git pull
```

```
dvc pull
```

۶. مفاهیم و دستورات اساسی

دستور	عملکرد و توضیحات
dvc init	مقداردهی و ایجاد تنظیمات اولیه DVC در پوشه پروژه.
dvc add [file]	شروع ردیابی یک فایل یا پوشه حجیم؛ تولید فایل متناظر .dvc.
dvc push	آپلود و انتقال فایل‌های ردیابی شده به فضای ذخیره‌سازی ابری (Remote).
dvc pull	دانلود و همگام‌سازی فایل‌های داده از Remote بر اساس متاداده‌های موجود.
dvc run	تعریف یک مرحله از خط‌لوله (Pipeline) و اجرای آن برای تولید خروجی.
dvc repro	بازتولید و اجرای هوشمند کل خط‌لوله (تنها بخش‌های تغییر یافته).
dvc status	مقایسه وضعیت داده‌های محلی با کش و ریموت جهت شناسایی تغییرات.
dvc metrics show	استخراج و نمایش مقادیر عددی ارزیابی مدل (مانند Accuracy یا Loss).
dvc checkout	بازیابی نسخه‌ای از داده‌ها که با فایل‌های .dvc فعلی مطابقت دارند.
dvc remote add	تعریف یک فضای ذخیره‌سازی جدید (مانند S3، Drive یا سرور محلی).

۷. کاربردهای اصلی

1. پروژه‌های بین‌تیمی: وقتی چندین دیتا ساینیتیست روی یک مدل کار می‌کنند و نیاز به اشتراک‌گذاری داده‌های حجیم دارند.

Versioning 2. مدل‌های سنگین: ذخیره نسخه‌های مختلف مدل‌های (Deep Learning مانند فایل‌های `h5` یا `pt`).

3. اتوماسیون یادگیری ماشین (MLOps) ایجاد خط‌لوله‌هایی که در صورت تغییر دیتاسِت، به طور خودکار مدل را دوباره آموزش می‌دهند.

4. تطبیق با استانداردها: در پروژه‌هایی که نیاز به شفافیت در منبع داده‌ها (Data Provenance) دارند.

۸. مزایا و معایب

مزایا:

سازگاری با Git: نیازی به یادگیری ابزار کاملاً متفاوت نیست؛ روند کاری بسیار شبیه Git است.
بهینه‌سازی فضا: داده‌های تکراری را دوباره ذخیره نمی‌کند. (Content-addressable storage)
عدم وابستگی به پلتفرم: برخلاف ابزارهایی مثل SageMaker ، در هر محیطی (Local, Cloud, On-premise) کار می‌کند.
رایگان و متن‌باز.

معایب:

پیچیدگی در پروژه‌های کوچک: برای پروژه‌هایی که داده‌های بسیار کمی دارند، ممکن است سربار ایجاد کند.
مدیریت Remote: تنظیمات اولیه Remote Storage مانند دسترسی‌های (S3) ممکن است برای افراد تازه‌کار کمی دشوار باشد.
تغییر فرهنگ تیمی: همه اعضا باید منضبط باشند و فراموش نکنند که همزمان با `git push` باید `dvc push` را هم انجام دهند.

۹. اکوسیستم و منابع یادگیری

ابزارهای مکمل:

CML (Continuous Machine Learning): برای اجرای CI/CD در پروژه‌های ML و گزارش‌دهی خودکار در GitHub/GitLab.

DVC Extension for VS Code: یک افزونه عالی برای مشاهده آزمایش‌ها و مدیریت داده‌ها در محیط ویرایشگر.

منابع مطالعه بیشتر:

1. مستندات رسمی

2. آموزش‌های ویدیویی: کانال یوتیوب Iterative.

3. کتاب: "Data Version Control" نوشته Dmitry Petrov (بنیان‌گذار DVC).

Docker 3-1

این یک گزارش جامع، حرفه‌ای و آموزشی در مورد Docker است که بر اساس استانداردهای مستندسازی فنی برای تیم شما تهیه شده است.

گزارش جامع مستندات Onboarding: تکنولوژی Docker

موضوع: پلتفرم کانتینرسازی (Containerization)

۱. مقدمه و تاریخچه

داکر (Docker) انقلابی در نحوه توسعه، اشتراک‌گذاری و اجرای اپلیکیشن‌ها ایجاد کرد. پیش از داکر، مشکل "اما روی سیستم من کار می‌کرد!" (It works on my machine) یکی از بزرگترین چالش‌های تیم‌های نرم‌افزاری بود.

سازنده: Solomon Hykes.

سال انتشار: اولین بار در سال ۲۰۱۳ به صورت متن‌باز (Open Source) معرفی شد.

هدف از ایجاد: هدف اصلی داکر، "بسته‌بندی" نرم‌افزار به همراه تمام وابستگی‌های آن (کتابخانه‌ها، تنظیمات و فایل‌های سیستم‌عامل) در یک واحد مستقل به نام Container بود تا در هر محیطی (توسعه، تست، تولید) دقیقاً به یک شکل اجرا شود.

سیر تکاملی: داکر ابتدا بر پایه تکنولوژی Linux Containers (LXC) بنا شد، اما بعداً موتور اختصاصی خود یعنی `libcontainer` را توسعه داد و اکنون به استاندارد بی‌قید و شرط دنیای DevOps و Microservices تبدیل شده است.

۲. توضیحات فنی و معماری

داکر بر پایه مفهوم Virtualization در سطح سیستم‌عامل کار می‌کند. برخلاف ماشین‌های مجازی (VMs) که هر کدام نیاز به یک سیستم‌عامل کامل (Guest OS) دارند، کانتینرهای داکر از هسته (Kernel) سیستم‌عامل میزبان به صورت مشترک استفاده می‌کنند.

اجزای اصلی معماری داکر (Client-Server Architecture):

1. Docker Daemon (dockerd): فرآیند پس‌زمینه‌ای که مدیریت اشیاء داکر مانند تصاویر، کانتینرها، شبکه‌ها و حجم‌ها را بر عهده دارد.
2. Docker Client: رابطی (CLI) که کاربر از طریق آن دستورات (مانند `docker run`) را به داکر دیمون ارسال می‌کند.
3. Docker Images: فایل‌های فقط‌خواندنی (Read-only) که قالب و دستورالعمل ساخت کانتینر هستند.
4. Docker Containers: نمونه‌های در حال اجرا (Instances) از یک Image.
5. Docker Registry: محل ذخیره‌سازی ایماژها (مانند Docker Hub).

۳. ویژگی‌های کلیدی

چرا داکر به ابزاری جدایی‌ناپذیر تبدیل شده است؟

Isolations (جداسازی): هر کانتینر در محیطی کاملاً ایزوله اجرا می‌شود و تداخلی با سایر کانتینرها یا سیستم میزبان ندارد.

Portability (قابلیت حمل): "یک بار بساز، هر جایی اجرا کن". ایماژی که روی لپ‌تاپ شماست، دقیقاً همان ایماژی است که روی سرورهای ابری اجرا می‌شود.

Lightweight (سبک بودن): به دلیل اشتراک هسته سیستم‌عامل، کانتینرها در عرض چند ثانیه بالا می‌آیند و منابع بسیار کمی (رم و سی‌پی‌یو) مصرف می‌کنند.

Version Control برای زیرساخت: ایماژهای داکر دارای تگ و نسخه هستند و می‌توان به راحتی به نسخه‌های قبلی بازگشت.

۴. پیش‌نیازها و نصب

برای شروع کار با داکر، باید Docker Desktop (برای ویندوز و مک) یا Docker Engine (برای لینوکس) را نصب کنید.

مراحل نصب:

ویندوز: نصب Docker Desktop. نیاز به فعال‌سازی ویژگی WSL 2 (Windows Subsystem for Linux) در ویندوز دارد.

مک: نصب Docker Desktop (نسخه‌های جداگانه برای تراشه‌های Intel و Apple Silicon).

لینوکس (Ubuntu):

```
sudo apt-get update
```

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

تست نصب: پس از نصب، ترمینال را باز کرده و دستور زیر را بزنید تا از صحت نصب مطمئن شوید:

```
`docker --version`
```

۵. راهنمای شروع سریع (Quick Start)

بیایید اولین کانتنر خود را بسازیم. فرض کنید می‌خواهیم یک وب‌سرور ساده با Nginx راه اندازی کنیم.

گام ۱: دریافت image و اجرا

دستور زیر را در ترمینال وارد کنید:

```
docker run -d -p 8080:80 --name my-webserver nginx
```

گام ۲: توضیح خط به خط دستور:

`docker run`: دستور اصلی برای ساخت و اجرای یک کانتنر.

`-d` (Detached): کانتینر را در پس زمینه اجرا می‌کند تا ترمینال آزاد بماند.

`-p 8080:80`: پورت ۸۰ کانتینر را به پورت ۸۰۸۰ سیستم شما متصل می‌کند. (دسترسی از طریق `localhost:8080`)

`--name my-webserver`: یک نام دلخواه برای کانتینر انتخاب می‌کند.

`nginx`: نام ایماژی است که از Docker Hub دانلود می‌شود.

گام ۳: ساخت یک پروژه شخصی (Dockerfile)

یک فایل به نام `Dockerfile` (بدون پسوند) بسازید و محتوای زیر را در آن قرار دهید:

```
dockerfile
```

1. مشخص کردن ایمیج پایه (مثلا پایتون)

```
FROM python:3.9-slim
```

2. تعیین پوشه کاری داخل کانتینر

```
WORKDIR /app
```

3. کپی کردن فایل‌های پروژه به داخل کانتینر

```
COPY . /app
```

4. اجرای دستور برای چاپ پیام

```
CMD ["python", "-c", "print('Hello, Docker Team!')"]
```

سپس برای ساخت و اجرا:

```
. docker build -t my-python-app
```

```
docker run my-python-app
```

5. کاربردهای اصلی

توسعه میکروسرویس‌ها: هر سرویس (دیتابیس، پنل مدیریت، API) در کانتینر جداگانه.
تست خودکار (CI/CD): ایجاد محیط‌های تست یکسان برای جلوگیری از خطا در خط لوله استقرار.
ساده‌سازی راه‌اندازی پروژه: تازه وارد تیم شده‌اید؟ فقط کافیست `docker-compose up` را بزنید تا کل پروژه (دیتابیس، ردیس، وب سرور و...) روی سیستم‌تان بالا بیاید.
ایزوله‌سازی دیتابیس‌ها: اجرای چندین نسخه از MySQL یا PostgreSQL بدون تداخل پورت‌ها و فایل‌ها.

6. مزایا و معایب

مزایا:

بهره‌وری بالا: استفاده بهینه از منابع سخت‌افزاری نسبت به VM.
ثبات: حذف تفاوت‌های محیط توسعه و تولید.
مقیاس‌پذیری: به راحتی می‌توان تعداد نمونه‌های یک کانتینر را افزایش داد.

معایب:

امنیت هسته مشترک: اگر هسته سیستم عامل میزبان آسیب‌پذیر باشد، کانتینرها هم ممکن است تحت تاثیر قرار گیرند.
پیچیدگی شبکه: مدیریت ارتباطات بین تعداد زیادی کانتینر نیاز به دانش شبکه دارد.
ذخیره‌سازی داده (Persistence): داده‌های داخل کانتینر با حذف آن پاک می‌شوند؛ لذا باید از مفهوم Volume برای ذخیره دائمی داده‌ها استفاده کرد.

7. اکوسیستم و منابع یادگیری

برای تسلط بیشتر بر داکر، ابزارها و منابع زیر توصیه می‌شود:

Docker Compose: ابزاری برای تعریف و اجرای اپلیکیشن‌های چند کانتینری (با استفاده از یک فایل `YAML`).

Docker Hub: مخزن رسمی ایماژها (مشابه GitHub برای کد).

Kubernetes (K8s): سیستم ارکستراسیون برای مدیریت هزاران کانتینر در مقیاس بزرگ.

Portainer: یک رابط کاربری گرافیکی (GUI) عالی برای مدیریت کانتینرها.

منابع پیشنهادی برای مطالعه:

1. مستندات رسمی داکر (docs.docker.com).

2. دوره Docker Mastery در Udemy.

3. وبسایت Play with Docker (برای تمرین آنلاین بدون نصب داکر).

prefect 4-1

گزارش جامع معرفی و آموزش ابزار Prefect (نسخه ۲,۰ و بالاتر)

۱. مقدمه و تاریخچه

در دنیای مهندسی داده و پردازش‌های ابری، مدیریت گردش کار (Workflow Orchestration) یکی از حیاتی‌ترین بخش‌هاست. **Prefect** ابزاری مدرن برای ارکستراسیون داده است که با هدف رفع محدودیت‌های ابزارهای قدیمی‌تری مانند Apache Airflow طراحی شده است.

- **سازنده و سال انتشار:** این پروژه توسط **Jeremiah Lowin** (که خود از مشارکت‌کنندگان سابق Airflow بود) در سال ۲۰۱۸ کلید خورد. شرکت Prefect Technologies در سال ۲۰۱۹ نسخه Core را معرفی کرد.
- **سیر تکاملی:** نسخه ۱,۰ (Prefect Core) بر پایه یک گراف جهت‌دار بدون دور (DAG) صلب بنا شده بود. اما با معرفی **Prefect 2.0** (معروف به Orion)، این ابزار متحول شد و مفهوم “Code as Workflows” را معرفی کرد که در آن دیگر نیازی به تعریف ساختارهای پیچیده DAG نیست و کدنویسی به سبک پایتون استاندارد انجام می‌شود.
- **هدف:** هدف اصلی Prefect، مدیریت “مهندسی منفی” (Negative Engineering) است؛ یعنی جابجایی تمرکز برنامه نویس از مدیریت خطاها، ریتراها (Retries) و لاگ‌ها به سمت نوشتن منطق اصلی برنامه.

۲. توضیحات فنی و معماری

Prefect یک سیستم ارکستراسیون “Python-native” است. برخلاف ابزارهای سنتی، Prefect بر پایه یک مدل **Hybrid** عمل می‌کند.

اجزای اصلی معماری:

1. **Prefect API (Server/Cloud):** مغز متفکر سیستم که وضعیت Flowها را ذخیره می‌کند، زمان‌بندی را مدیریت کرده و داشبورد UI را ارائه می‌دهد.
2. **Database:** برای ذخیره متادیتاها (معمولاً PostgreSQL یا SQLite).
3. **Workers & Work Pools:** این بخش مسئول اختصاص منابع برای اجرای کدهاست. Workerها به API متصل شده و منتظر دریافت دستور اجرا می‌مانند.

4. **Tasks & Flows**: کوچکترین واحدهای منطقی در Prefect هستند که در بخش مفاهیم بررسی می‌شوند.

نحوه عملکرد:

در معماری جدید Prefect، کد شما لزوماً نباید در سرور Prefect مستقر باشد. کد می‌تواند در محیط لوکال، کانتینر Docker یا کلاستر Kubernetes اجرا شود و فقط وضعیت‌های (States) خود را به سرور گزارش دهد. این امنیت داده‌ها را تضمین می‌کند زیرا داده‌های حساس از محیط شما خارج نمی‌شوند.

۳. ویژگی‌های کلیدی

چرا تیم ما باید از Prefect استفاده کند؟

- **Dynamic Workflows**: برخلاف Airflow، در Prefect جریان‌ها کاملاً دینامیک هستند. می‌توانید از دستورات if/else و loops معمولی پایتون استفاده کنید.
- **Pythonic Approach**: هر تابع پایتونی را می‌توان تنها با یک Decorator ساده به یک Task تبدیل کرد.
- **Observability**: یک پنل مدیریت بسیار قدرتمند و زیبا برای مشاهده وضعیت اجرای پروژه‌ها، زمان‌بندی‌ها و لاگ‌ها در لحظه.
- **Error Handling**: قابلیت تعریف خودکار retries در صورت شکست خوردن یک تسک با فواصل زمانی مشخص (Exponential Backoff).
- **Caching**: قابلیت ذخیره خروجی تسک‌ها (Caching) برای جلوگیری از محاسبات تکراری و صرفه‌جویی در هزینه و زمان.

۴. پیش‌نیازها و نصب

برای شروع کار با Prefect، سیستم شما باید حداقل‌های زیر را داشته باشد:

پیش‌نیازها:

- پایتون نسخه ۳,۸ یا بالاتر.
- دسترسی به اینترنت برای نصب پکیج‌ها و اتصال به پنل (در صورت استفاده از نسخه Cloud).

- نصب بودن pip (مدیریت پکیج پایتون).

مراحل نصب:

در محیط ایزوله (Virtual Environment) دستور زیر را اجرا کنید:

```
python -m venv prefect-env  
source prefect-env/bin/activate
```

```
pip install -U prefect
```

برای تایید نصب، دستور زیر را در ترمینال بزنید:

```
prefect version
```

۵. راهنمای شروع سریع (Quick Start)

در Prefect، هر اسکریپت از دو بخش اصلی تشکیل شده است: **Flow** (جریان اصلی) و **Task** (واحدهای کاری).

نمونه کد Hello World:

یک فایل به نام `my_first_flow.py` بسازید:

```
from prefect import flow, task  
  
@task  
def get_message():  
    return "Hello, Team! Welcome to Prefect."  
  
@task  
def print_message(msg):  
    print(msg)  
  
@flow(name="My First Onboarding Flow")
```

```
def main_flow():
    message = get_message()
    print_message(message)

if __name__ == "__main__":
    main_flow()
```

توضیح خط به خط:

1. **@task**: این دکوراتور تابع معمولی پایتون را به یک واحد کاری Prefect تبدیل می‌کند که قابلیت ردیابی، کش شدن و ریترا دارد.

2. **@flow**: این دکوراتور ورودی اصلی برنامه شماست. مانند یک کانستینر عمل می‌کند که تسک‌ها را سازماندهی می‌کند. نامگذاری آن در UI نمایش داده می‌شود.

3. اجرا: وقتی `main_flow()` فراخوانی می‌شود، Prefect یک "Flow Run" ایجاد کرده و تمام مراحل را لاگ برداری می‌کند.

برای مشاهده داشبورد در محیط لوکال، دستور زیر را در یک ترمینال جداگانه بزنید:

```
prefect server start
```

۶. مفاهیم و دستورات اساسی

در جدول زیر، دستورات و اصطلاحاتی که هر عضو تیم باید بداند آورده شده است:

عملکرد	دستور / مفهوم
راه‌اندازی پنل مدیریتی (UI) و API روی سیستم محلی (localhost:4200).	prefect server start
مشاهده تنظیمات فعلی محیط Prefect.	prefect config view
فرآیند بسته‌بندی فلو برای اجرای زمان‌بندی شده و مدیریت شده توسط سرور.	Deployment
مکانیزم صف‌بندی که فلوها را به Workerهای مناسب هدایت می‌کند.	Work Pool

عملکرد	دستور / مفهوم
وضعیت فعلی فلو یا تسک (Scheduled, Running, Completed, Failed).	State
خروجی های انسانی مانند جداول یا لینک ها که در UI قابل مشاهده هستند.	Artifacts

۷. کاربردهای اصلی

Prefect در چه پروژه هایی در تیم ما به کار می آید؟

1. خط لوله های ETL/ELT: استخراج داده از دیتابیس های مختلف، تبدیل آن ها و بارگذاری در انبار داده (Data Warehouse).
2. یادگیری ماشین (MLOps): خودکارسازی فرآیند آموزش مدل، ارزیابی و استقرار (Deployment) مدل های هوش مصنوعی.
3. گزارش دهی خودکار: اجرای کوئری های سنگین در ساعت های کم بار شب و ارسال نتایج به ایمیل یا Slack.
4. مدیریت زیرساخت: اجرای اسکریپت های دوره ای برای پاکسازی دیتابیس یا بررسی سلامت سرویس ها.

۸. مزایا و معایب (دید واقع بینانه)

مزایا:

- توسعه سریع: اگر پایتون بلد باشید، در کمتر از یک ساعت اولین فلو خود را می سازید.
- انعطاف پذیری فوق العاده: برخلاف Airflow، محدود به ساختار درختی ثابت نیستید.
- UI مدرن: داشبورد آن بسیار کاربر پسندتر از رقبا است.
- Community قوی: مستندات عالی و جامعه کاربری فعال در دیسکورد و گیت هاب.

معایب:

- تغییرات نسخه ها: تغییرات از نسخه ۱ به ۲ بسیار زیاد بود و ممکن است برخی آموزش های قدیمی در اینترنت منسوخ شده باشند.

- پیچیدگی در مقیاس بزرگ: مدیریت چندین Work Pool و Worker در محیط‌های پیچیده ابری نیاز به دانش DevOps دارد.

۹. اکوسیستم و منابع یادگیری

ابزارهای مکمل:

- **Prefect Cloud**: نسخه ابری و مدیریت شده (SaaS) که نیاز به نگهداری سرور را از بین می‌برد.
- **Prefect Integrations (Blocks)**: کتابخانه‌های آماده برای اتصال به AWS (S3), GCP, Snowflake, GitHub, Slack و غیره. برای نصب مثلاً: `pip install prefect-aws`.

منابع پیشنهادی برای یادگیری بیشتر:

1. **Documentation**: مستندات رسمی در docs.prefect.io (بهترین منبع).
2. **Prefect Discourse**: تالارگفت‌وگو برای حل مشکلات فنی.
3. **YouTube**: کانال رسمی Prefect برای مشاهده ویدیوها و آموزش‌های تصویری.

نکته ای شایان توجه که باید به آن توجه داشت این است که محتوای درج شده در این گزارش و سایر مستندات فنی پروژه، حاصل فرآیند تحقیق و توسعه (D&R) مستمر تیم فنی است. جهت تدوین این راهنماها، علاوه بر مستندات رسمی هر ابزار، از منابع چندگانه و متنوعی جهت بومی سازی و انطباق با نیازهای اختصاصی پروژه استفاده شده است. فرآیند جمع آوری دانش و تدوین این مستندات بر پایه محورهای زیر استوار بوده است:

۱. **تحلیل منابع چندرسانه‌ای:** بهره‌گیری از دوره‌های آموزشی تخصصی و سمینارهای فنی در پلتفرم‌های اشتراک ویدیو نظیر یوتیوب (YouTube) جهت درک عمیق پیاده‌سازی‌های عملی.

۲. **جستجوهای تخصصی در وب:** استخراج الگوهای بهینه (Best Practices) از طریق موتورهای جستجو و بررسی مقالات تحلیلی در وبسایت‌های مرجع مهندسی نرم‌افزار.

با توجه به گستردگی ابزارهای مورد استفاده در تیم فنی (Tech Stack) پروژه و حجم انبوه منابع بررسی شده، ذکر تمامی جزئیات، پیوندها (Links) و مراجع در این فایل و سایر اسناد مشابه میسر نبوده است. این مجموعه مستندات صرفاً به عنوان یک «چهارچوب عملیاتی» و چکیده‌ای از یافته‌های کلیدی تدوین شده است تا فرآیند یادگیری اعضایی از تیم که با ابزارها آشنایی ندارند با سرعت و دقت بیشتری صورت پذیرد.

ممنون از توجه شما

تیم تحقیق

2. صور تجلسات

2-1 صورت جلسه هماهنگی پروژه MLOps (جلسه اول - هفته اول)

مشخصات جلسه:

تاریخ و زمان: 1405/02/29 - 60 دقیقه

پلتفرم برگزاری: جلسه آنلاین در پیام رسان بله

حاضرین: محمدرضا باقرنژادمیری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

۱. دستور جلسه

- تبیین استراتژی کلی: حرکت از «مدل سازی صرف» به سمت «زیرساخت توزیع شده».
- بررسی منابع آموزشی و مستندات آنلاین برای همگام سازی دانش اعضای تیم.
- ایجاد لیست تسک های هفته اول و داوطلب شدن اعضا برای انجام آنها.

۲. گزارش بخش مطالعاتی و منابع (مسئول: کیارش رحمانی)

مستندات مرجع:

تحلیل فایل Mlops.pdf: تأکید بر چرخه کامل (Collection -> Preprocessing -> Training -> Deployment -> Monitoring). مقرر شد تمرکز تیم به جای پیچیدگی معماری مدل، روی اتوماسیون و نگهداری قابل اطمینان باشد.

مستندات آنلاین: بررسی داکيومنت های مربوط به ابزارهای پیشنهادی (مانند docker یا DVC) برای مدیریت آزمایش ها.

خروجی کلیدی: برای هفته های آتی، اولویت با طراحی Pipeline است تا صرفاً اجرای یک اسکریپت ساده پایتون.

۳. وضعیت تسک‌ها (تسک‌بندی داوطلبانه)

در این بخش، تسک‌های استخراج شده از نقشه راه پروژه لیست شد و اعضا داوطلبانه مسئولیت آن‌ها را پذیرفتند:

ردیف	شرح تسک	وضعیت	مسئول
1	تدوین مستندات نهایی تحقیق و اشتراک منابع در «بله»	پایان یافته	کیارش رحمانی
2	انتخاب و تست اولیه ابزار کنترل نسخه داده (Data Versioning)	پایان یافته	معصومه مختاری
3	جستجو برای دیتاست مناسب NLP	پایان یافته	معصومه مختاری
4	طراحی اولیه معماری خط لوله (Pipeline)	پایان یافته	تمام اعضای تیم
5	اجرا و نصب dvc	پایان یافته	معصومه مختاری
6	نصب docker و ایجاد image و فایل‌های مربوطه	پایان یافته	محمدرضا باقرنژادمیری
	تدوین مستندات فنی و چالش‌ها	پایان یافته	یاسمن دادفر

۴. مصوبات و خروجی‌های جلسه

مقرر شد تمامی ارتباطات فنی و ارسال فایل‌ها در گروه بله متمرکز باشد.

تسک‌های بدون مسئول در گروه اعلام شده تا سایر اعضا تا ۴۸ ساعت آینده داوطلب شوند.

جلسه بعدی جهت بررسی پیشرفت تسک‌های داوطلبانه برگزار خواهد شد.

تنظیم‌کننده گزارش: کیارش رحمانی

منابع مورد ارجاع: فایل راهنمای تمرین عملی MLOps + مستندات آنلاین استخراج شده.

2-2 صورت جلسه هماهنگی پروژه MLOps (جلسه دوم و سوم - هفته اول)

مشخصات جلسه:

تاریخ و زمان: 1405/03/01 (جلسه پیگیری) - ۶۰ دقیقه

پلتفرم برگزاری: هماهنگی مستمر در پیام رسان بله و تماس های تلفنی اعضا

حاضرین: محمدرضا باقرنژادمیری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

۱. دستور جلسه

- بررسی آخرین وضعیت پیشرفت تسک های هفته اول و رفع چالش های فنی از طریق تماس های تلفنی.
- همگام سازی تمامی اعضا در خصوص خروجی های به دست آمده و تسک های جدید ایجاد شده.
- برنامه ریزی و تنظیم زمان جلسه آنلاین اصلی برای فاز دوم در هفته آینده.

۲. گزارش بخش مطالعاتی و منابع (مسئول: کیارش رحمانی)

مستندات مرجع:

تحلیل فایل **Mlops.pdf**: با تمرکز بر بخش زیرساخت، مطالعه روی نحوه اتصال اجزای خط لوله (Pipeline) انجام شد تا اطمینان حاصل شود که تمامی مراحل از پیش پردازش تا استقرار با استانداردهای فایل مطابقت دارد.

مستندات آنلاین و کتاب: مطالعه داک های پیشرفته تر Docker برای بهینه سازی Image ها و بررسی منابع مربوط به مدیریت سناریوهای NLP در محیط عملیاتی.

خروجی کلیدی: آماده سازی مستندات راهنما برای اعضا جهت یکپارچه سازی ابزارهای نصب شده (Docker و DVC) در فاز بعدی.

۳. وضعیت تسک ها (تسک بندی داوطلبانه)

در این مرحله، علاوه بر تسک های قبلی، موارد جدیدی که در حین کار ایجاد شده بود در گروه تقسیم و نهایی شد.

۴. مصوبات و خروجی های جلسه

- با توجه به هماهنگی های صورت گرفته در پیام رسان بله و تماس های تلفنی، تمامی اعضا به طور کامل در جریان جزئیات پیشرفت و تغییرات قرار گرفتند.

- تسک‌های جدیدی که در حین اجرای پروژه ایجاد شده بود، بلافاصله در گروه بله مطرح و مسئولیت آن‌ها مشخص و انجام شد.

- مقرر شد جلسه آنلاین بعدی جهت ورود به فاز اجرایی مدل، برای هفته آینده هماهنگ و برگزار شود.

تنظیم‌کننده گزارش: کیارش رحمانی

